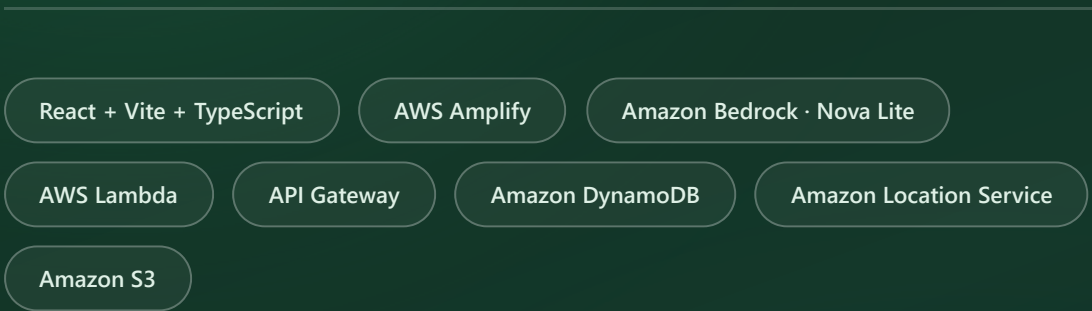


SmartSort

Architecture Evolution & Final Technical Plan

AI-Powered Garbage Segregation & Reuse Mapping

Original Concept → Technical Refinement → Final AWS Architecture



Prepared for: hackathon judges · mentors · engineering reviewFrontend phase built & verified · Backend specified

Executive Summary

Problem

Households generate recyclable waste daily but reliably fail three steps: they cannot identify *what material* an item is, they have *no discovery mechanism* for facilities that accept it, and they cannot see *whether it has value*. The result is predictable — recyclables land in landfills and potential income is lost. Classification alone does not close this loop; a person holding waste needs a decision, not a label.

Solution

SmartSort is a **mobile-first responsive web application** — no app install, opened via an HTTPS URL. The user photographs a waste item; an AI service identifies the material; the user confirms or corrects it; and the system returns ranked nearby facilities with distance, operating hours, verification status, and an indicative value in ₹/kg. Every payout is explicitly labeled *“Indicative market estimate — not an offer.”*

Core product flow

Capture → Compress → Classify → Confirm → Locate → Match → Rank → Display

Core differentiation

SmartSort is not merely a waste-classification system.
Its product value is the full bridge: **Waste Identification + Facility Discovery + Distance + Estimated Value**. The classification is the entry point; the discovery of where waste becomes worth something is the product.

Current status

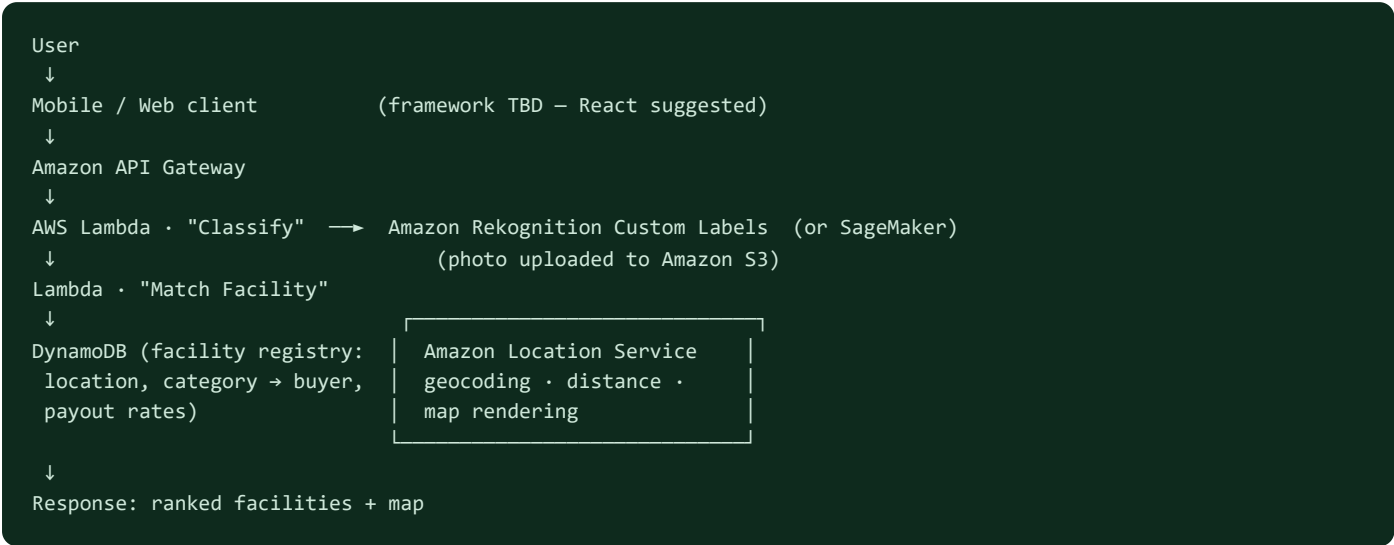
Layer	Status
Frontend (React + Vite + TS)	Built, unit-tested (18/18), flow verified end-to-end in browser; mock-backed API layer behind a swappable interface
Backend (AWS)	Fully specified (SAM template, Lambda handlers, Bedrock prompt/schema, IAM); not yet deployed
Data	14-facility curated Bengaluru demo registry finalized; payouts labeled as estimates

Honesty rules honored throughout this document
No verified AI accuracy, guaranteed latency, guaranteed payout, real-time facility availability, or production readiness is claimed anywhere. Demo and estimated data are labeled as such.

The Original Plan (as proposed)

Recorded faithfully from the initial project proposal (Sept 2026). History is not rewritten here — later sections analyze it.

Proposed flow



Layer	Original choice
AI classification	Amazon Rekognition Custom Labels (default) or SageMaker custom model — decided by team before build; Claude recommended Custom Labels for speed
Storage	Amazon S3 for raw uploaded waste photos
Compute	AWS Lambda — stateless classification trigger + facility matching
API	Amazon API Gateway — REST endpoints to client
Database	Amazon DynamoDB — facility registry (location, accepted materials, payout rates)
Location	Amazon Location Service — geocoding, distance calculation, map rendering
Hosting	AWS Amplify or S3 + CloudFront (web client)
Auth (optional)	Amazon Cognito — user accounts, saved locations/history
Frontend	Mobile or web client, framework TBD (React suggested for team familiarity)
Demo data	Open decision — manual entry vs scraped/public data

Original milestones

Day 1 decisions + AWS setup · Day 2 upload pipeline · Day 3 classification integrated · Day 4 Amazon Location matching · Day 5 frontend map + E2E demo · Day 6 polish, testing, submission.

What the original plan got right

Serverless-first shape (API Gateway → Lambda → DynamoDB) · DynamoDB as facility source of truth · Amazon Location for maps/geocoding · React-family frontend · S3 for images · a working end-to-end demo as the goal. The final architecture retains all of these bones.

SmartSort — Architecture Evolution3 / 22

Why the Original Plan Needed Refinement

Findings from three review passes (independent research review → Claude re-evaluation → Claude Opus refinement → GLM consolidation).
Every row traces to a concrete change shown on pages 6–12.

Problem	Why it mattered	Consequence in final plan
AI training dependency Custom Labels / SageMaker	Custom Labels needs ~15–20+ labeled images per category and bills \$4/hour while a model is running ; SageMaker adds pipeline complexity. A dataset-collection task sat on the hackathon critical path.	MODIFY → Bedrock Nova Lite: zero training, zero hosting, per-request pricing
Unnecessary AWS complexity	Cognito accounts, hand-managed CloudFront, and an async S3-trigger pipeline each added setup and failure modes without serving the 5-day MVP.	REMOVED → Cognito/CloudFront cut; synchronous request/response; S3 becomes fire-and-forget archive
Image payload size	Modern phone photos run 3–8 MB — against API Gateway's payload ceiling, mobile bandwidth, and Bedrock token cost.	ADDED → client-side compression (≤ 1024 px, $\approx \leq 800$ KB, best-effort)
Fragile JSON parsing	Free-form “respond with JSON” + regex parsing breaks silently on chatty model output.	ADDED → Bedrock Converse API with tool/function schema + mandatory runtime validation
Human override architecture	A single combined endpoint meant a user-corrected category would re-invoke Bedrock — wasted cost/latency and conceptually wrong.	SPLIT → /classify-and-match + /match-facilities; AI runs at most once per image
Ranking weights summed to 1.1	$0.7 + 0.3 + 0.1 = 1.1$ made scores non-interpretable as shares and diluted the verification bonus.	REBALANCED → 0.6 / 0.3 / 0.1 (sums to 1.0), unit-tested
Facility dataset inconsistency	Draft used 12 facilities; “ ≥ 2 verified per category” coverage was arithmetically marginal; demo city wavered between Kolkata and Bengaluru.	STANDARDIZED → 14 curated Bengaluru facilities; coverage proof per category
Confidence terminology	Model confidence was at risk of being shown as “accuracy” — an unverifiable claim to judges.	RENAMED → “AI signal”, never accuracy/probability
Missing operating hours	A drop-off recommendation without hours can send users to a closed facility — core-flow defect, not polish.	ADDED → operating_hours in schema + seed data + UI
Demo-location inconsistency	Fallback coordinates differed between documents; demo scripts need one fixed point.	FIXED → Bengaluru 12.9716, 77.5946, always labeled “Demo location”
Unclear MVP boundaries	Login, payments, marketplace concepts floated near the MVP; risk of scope creep in 5 days.	LOCKED → explicit out-of-scope list; trading-platform concepts documented as future vision only

Architecture Evolution Timeline

STAGE 1

Original Proposal

Key decision: serverless AWS shape; classification service undecided between Rekognition Custom Labels and SageMaker.

Problem later found: training dependency, per-hour model hosting cost, unbounded scope. **Result:** everything downstream re-examined.

STAGE 2

Technical Review (research-based, document-grounded)

Key decision: against current AWS docs — Custom Labels' real advantage is managed hosting, not “minimal data”; its \$4/hr running-model billing is a live-demo hazard; Bedrock multimodal (Nova) available in ap-south-1 at per-request pricing was never evaluated.

Problem discovered: highest-risk component (custom model training) sat on the critical path. **Result:** Bedrock Nova proposed as primary classifier behind a stable Lambda abstraction; Cognito and async pipelines flagged as unnecessary; dataset = curated facilities, no scraping.

STAGE 3

Claude Re-evaluation

Key decision: agreed with hybrid facility/map architecture (DynamoDB source of truth + Location for maps/geocoding) and React web frontend; sharpened service roles (Maps render, Places geocode, Haversine ranks, Routes optional).

Problem discovered: classification decision still defaulted toward Custom Labels; single-endpoint design persisted. **Result:** partial convergence — map and frontend decisions locked.

STAGE 4

Claude Opus Refinement

Key decisions: Bedrock Converse API with tool schema (vs regex parsing); weights rebalanced 0.6/0.3/0.1; image compression added; human override made a first-class flow; Bengaluru standardized as demo city; 14-facility registry; confidence renamed “AI signal”.

Problem discovered: earlier plans under-specified the override path and payload safety. **Result:** implementation-ready spec with acceptance criteria.

STAGE 5 — CURRENT

Final GLM Consolidated Plan (authoritative)

Key decisions: frontend-first build with swappable mock API; two Lambdas (classify-match / match-facilities); runtime validation chain regardless of schema; ranking formula unit-tested with documented crossover; fallback matrix complete; demo registry with 7×2 verified coverage proof.

Status: frontend implemented and verified (18/18 tests, E2E browser flow); backend fully specified, pending deployment. **Result:** the plan this document describes.

Pattern across all five stages

Each stage *removed* risk rather than adding features: fewer services on the demo path, fewer unverifiable claims, more failure modes handled by design. The architecture got simpler as it got better.

Before vs After — Master Comparison

Area	Earlier plan	Current plan	Why changed → benefit
Product positioning	"AI garbage segregation" app; classification centric	Waste-to-value discovery bridge: identify → locate → value → discover	Classification is entry point, not product → clearer pitch, clearer scope
Frontend	Mobile or web, framework TBD; Cognito optional	React + Vite + TypeScript responsive web on Amplify; mobile-first; no native app tech	Camera+GPS work on phone browsers over HTTPS → no store, no install, demo-ready
AI	Rekognition Custom Labels or SageMaker (train on own photos)	Bedrock Nova Lite via Converse API + tool schema; Custom Labels post-hackathon	Zero training/hosting on critical path → demo cannot be blocked by an ML pipeline
API	POST /classify-and-match (single, combined)	+ POST /match-facilities (override/manual path, no AI)	AI at most once per image → lower cost/latency, correct HITL architecture
Image processing	Phone image → backend as-is	Browser compression ≤ 1024 px, $\approx \leq 800$ KB best-effort, graceful fallback	Smaller payloads, faster uploads, lower Bedrock token cost → safer on venue Wi-Fi
Database	DynamoDB (shape unspecified)	DynamoDB PAY_PER_REQUEST; full schema incl. payout_basis, operating_hours, source, updated_at	Trust/audit fields first-class → honest UI, seedable registry
Facilities	Open decision: manual vs scraped	14 curated Bengaluru facilities; ≥ 2 verified per category; no scraping	Coverage proven per category → empty-result demo failure designed out
Location	Location Service broadly (geocode/distance/render)	Maps render · Places geocode at curation · Haversine ranks · Routes optional top-3	Each API used for what it actually does → no misuse of POI search
Ranking	$0.7/0.3/0.1 = 1.1$ (unnormalized)	$0.6/0.3/0.1 = 1.0$; per-set payout normalization; 5 km boundary; ties by distance	Interpretable shares, unit-tested → explainable to judges in one sentence
Map	Map rendering (library unspecified)	MapLibre GL JS behind abstraction; dev-safe schematic now; list never depends on map	List-first reliability → map failure cannot break the product
S3	Upload target in pipeline (in path)	Fire-and-forget archive of classified photos; 90-day lifecycle; public-access blocked	Storage can never fail the user flow → future training corpus for free
Authentication	Cognito (optional)	None in MVP; reserved post-hackathon	Removes setup + token/CORS complexity → 5-day feasibility
Infrastructure	Mixed console setup; CloudFront optional	AWS SAM template; one stack: API GW + 2 Lambdas + DynamoDB + S3 + Location + IAM	One deploy, reproducible → reviewable by mentors in minutes
Security	Unspecified	Least-privilege per-Lambda IAM; scoped S3 puts; CORS locked post-Day-1; API-key-restricted map; validation at every boundary	Provable minimal permissions → defensible in review
Fallback strategy	Not designed	Full matrix: AI failure, invalid tool output, GPS denial, wrong class, empty result, map/network failure, oversized image, S3 failure	Demo recovers from every identified failure → reliability by design

Testing	Unspecified	18 unit tests (formula, Haversine, compression, determinism) + E2E flow verification; AI evaluation planned on internal test set	Claims are backed by tests → no hand-waving under questioning
MVP scope	Loosely bounded; payments/marketplace adjacent	Locked: identify/confirm/locate/match/rank/display only; trading platform documented as long-term vision	Zero scope creep on critical path → shippable in 5 days
SmartSort — Architecture Evolution			6 / 22

AI Architecture Evolution

Earlier — custom-trained vision

Rekognition Custom Labels (primary candidate) or a **SageMaker** custom CNN.

- Collect & label ~15–20+ photos per category (7 categories) under demo-like conditions
- Train, then **start/stop a hosted model** — Custom Labels bills ≈\$4/hour while running
- SageMaker: notebook → training job → endpoint; strongest ceiling, heaviest pipeline
- Single-class classifier: cluttered bins and multi-object scenes are out of scope
- Post-hackathon evolution path (per original proposal), re-evaluated with real usage data

Current — Amazon Bedrock Nova Lite

- **Zero training**: categories defined in prompt/schema; no dataset collection on the critical path
- **Zero hosting**: on-demand API, per-request pricing (fractions of a cent per photo)
- Multimodal: robust to clutter, multiple objects, messy real-world photos
- Returns category + confidence signal + one-line rationale (material-composition prompt)
- Available in ap-south-1 (Mumbai) — matches the pinned region

How classification is invoked

Bedrock Converse API

```
toolConfig: classify_waste { category: enum[7], confidence: 0..1, rationale }
toolChoice: forced → { tool: "classify_waste" }
image:      base64 JPEG (browser-compressed) + material-composition prompt
temperature: 0 (deterministic)
```

Validation chain (mandatory — schema ≠ guarantee):

1. toolUse block present? → else parse text fallback
2. category ∈ enum? → else reject
3. confidence ∈ [0,1] (clamped)? → else 0
4. failure path → retry ×2 → fallback model ID → manual category chips

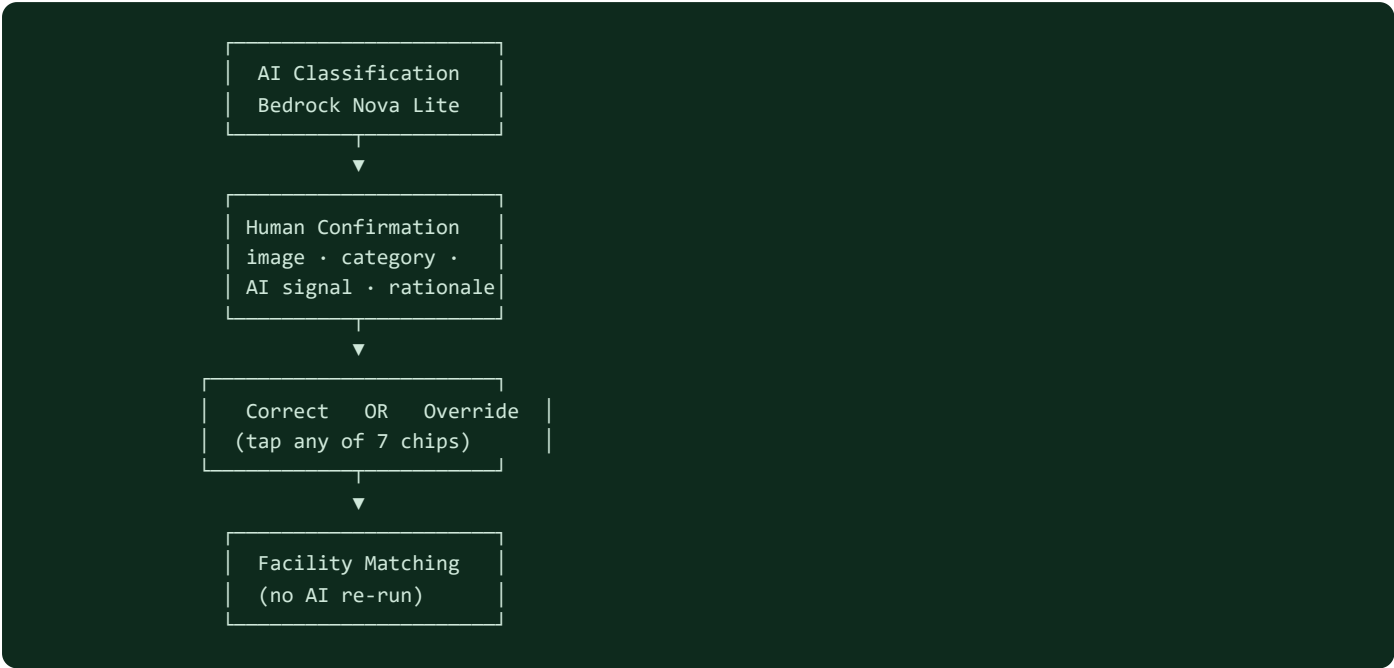
Required caveat

Bedrock's model-generated confidence is an **AI signal**, not a calibrated probability and not independently verified accuracy. The UI shows "AI signal: 87%" and always lets the user correct the category. No generalized accuracy claim is made or displayable.

Why this is the right hackathon trade

The MVP's risk was never model *quality* — it was model *operations*: collecting a labeled dataset by deadline, hosting a model reliably, and paying while idle. Bedrock removes all three. The original proposal itself anticipated this evolution ("move toward SageMaker/Bedrock once enough labeled data exists") — the review simply inverted the order: start with Bedrock, earn the custom model later using the photo corpus the app archives to S3.

Human-in-the-Loop Classification



Why waste demands a human gate

Real-world case	Why the AI alone is insufficient
Dirty plastic	Food residue changes texture/translucency; material is still plastic — a human glance resolves instantly
Damaged items	Crushed cans or torn paper lose their canonical silhouette
Ambiguous materials	Coated paper cups (paper? plastic? both) genuinely confuse material classification
Multiple objects	A drawer of mixed items has no single “correct” class — the user picks what they're actually disposing
Unclear images	Blur/low light → low AI signal; the flow must continue, not dead-end

Design rule

Confirming or overriding a category routes **directly to facility matching** — the backend never re-runs Bedrock for a human-corrected category (lower latency, lower cost, correct semantics). Low AI signal (< 0.6) shows a gentle “Not completely sure — please confirm the material” nudge; it never alarms and never blocks. The correction interface doubles as the manual path when the AI is entirely unavailable.

Startup story, not just UX: every user correction is a labeled training example. The override stream (archived with the S3 photo corpus) is exactly the dataset a future fine-tuned classifier would need — the product improves by being used.

SECTION 08

API Evolution

Earlier — one combined endpoint

```
POST /classify-and-match
{ imageBase64, lat, lng }
↓
Bedrock → match → response
```

A user-corrected category forced a full re-classification: wasted AI cost, wasted latency, and conceptually wrong (the AI already did its job).

Current — separation of concerns

```
POST /classify-and-match
{ imageBase64, lat, lng }
→ AI + matching (once per image)

POST /match-facilities
{ category, lat, lng }
→ matching ONLY (no AI)

GET /facilities    (registry/debug)
GET /health       (pre-demo check)
```

Core invariant

AI classification happens at most once per image. Confirmation/override → /match-facilities. Verifiable in logs: one Bedrock invoke per photo, regardless of how many times the user changes category.

Benefit	Why it matters
Lower cost	Bedrock invoked only for genuinely new images; overrides are pure compute (~100 ms, no AI)
Lower latency	Correction → results in a fraction of a second instead of a full AI round-trip
Cleaner separation	Classification Lambda holds Bedrock permission; matching Lambda is DynamoDB-only — provable cost isolation per route
Easier testing	Matching logic testable without any AI dependency; classification testable with canned images
Correct HITL architecture	The human's category decision is authoritative input to matching, not a trigger to re-guess

Image Pipeline

Earlier

```
Phone Image  
(3-8 MB typical)  
↓  
API Gateway  
↓  
Lambda → Bedrock
```

Risks: API Gateway payload ceiling, slow uploads on mobile data, oversized Bedrock image tokens, venue-Wi-Fi multipart failures.

Current

```
Phone Image  
↓  
Browser Compression  
(canvas, iOS-Safari-safe)  
↓ ≤1024 px long edge  
↓ JPEG q0.8 → step to q0.5  
↓ ≈800 KB target  
API Gateway  
↓  
Lambda  
↓  
Bedrock
```

Why compression helps

- **Payload safety:** 10 MP phone JPEGs can exceed limits; compressed output sits far under them
- **Mobile bandwidth:** an ~800 KB upload is seconds on cellular; 6 MB is a demo hazard
- **Bedrock efficiency:** smaller images mean fewer image tokens — cheaper and faster inference
- **Quality is preserved for the task:** 1024 px is ample for material classification

Honest limitation

Client-side compression is **best-effort**. Quality-stepping reduces size toward ~800 KB, but it does **not mathematically guarantee** every payload will be accepted: pathological images (noise-heavy textures compress poorly) can remain larger, and browsers without canvas support skip compression entirely (the original bytes are sent as fallback). Server-side payload limits remain the final guard. This is explicitly accepted as a demo-day tradeoff, not hidden.

Implementation note: canvas resize + JPEG re-encode in the browser (`document.createElement('canvas')` chosen over `OffscreenCanvas` for iOS Safari compatibility). Compression failure never blocks the flow — the app proceeds with original bytes and logs a warning.

SECTION 10

Ranking Evolution

Earlier formula

$$0.7 \times \text{Distance} + 0.3 \times \text{Payout} + 0.1 \times \text{Verification} = 1.1$$

Weights summing to 1.1 meant no term was an interpretable share, the maximum achievable score was 1.1, and the “verification bonus” was effectively diluted. Caught in review; fixed, not silently patched.

Current formula

$$0.6 \times \text{Distance} + 0.3 \times \text{Payout} + 0.1 \times \text{Verification} = 1.0$$

distance_score = max(0, 1 - distance_km / 5) · payout_score = payout / max_payout_among_eligible · verification = 1|0

Component	Design decision
Distance normalization	Linear to a 5 km boundary, then floored at 0 — facilities beyond 5 km are <i>ranked lower, never hidden</i> (flagged “>5 km” in UI)
Payout normalization	Divided by the best payout <i>among eligible facilities for that query</i> — scale-free across categories, self-adjusting per result set; denominator floored at 1 (no div-by-zero for free municipal drop-offs)
Verification bonus	Bonus, <i>not a filter</i> — unverified facilities rank lower and carry a visible badge but are never suppressed
Raw scores	Internal only. Users see ordered results, distance, estimated value, badges — never “0.756”, which means nothing to a household
Crossover behavior	Documented and unit-tested: at these weights, a large payout gap <i>can</i> legitimately overcome a short distance gap (~2 km) — correct for bulk quantities, and the roadmap story for weight personalization

Worked example (unit-tested)

Scenario: verified facilities, plastic, max eligible payout = ₹30/kg

A: 1.0 km, ₹20/kg

distance = 1 - 1.0/5 = 0.80 → 0.6 × 0.80 = 0.480

payout = 20/30 = 0.667 → 0.3 × 0.667 ≈ 0.200

verified = 1 → 0.1 × 1 = 0.100

SCORE = 0.780

◀ ranks first

B: 2.0 km, ₹30/kg

distance = 1 - 2.0/5 = 0.60 → 0.6 × 0.60 = 0.360

payout = 30/30 = 1.00 → 0.3 × 1.00 = 0.300

verified = 1 → 0.1 × 1 = 0.100

SCORE = 0.760

A wins: at household scale (a few kg), ₹10/kg more does not justify double the travel. Payout CAN win – but only over larger gaps or with bulk quantities (the documented crossover at distance-weight ≈ 0.61).

SmartSort — Architecture Evolution

11 / 22

Facility Data Evolution

12 facilities (draft, mixed city) → 14 curated facilities (Bengaluru, standardized)

Aspect	Decision
Demo city standardized	Bengaluru (12.9716, 77.5946) — earlier documents wavered (Kolkata vs Bengaluru); one fixed demo point with a labeled "Use Demo Location" fallback
Why 14	Minimum that satisfies the coverage rule with margin: every one of the 7 categories accepted by ≥ 2 verified facilities within ~4.5 km . At 12, several categories sat at exactly 2 — one bad record from an empty result on stage
Category coverage	plastic 5 · paper 5 · metal 5 · glass 3 · e-waste 3 · organic 2 · other 2 (verified facilities) — incl. one municipal drop-off accepting all categories
Verified / unverified	13 verified (field-visit / municipal-listing sources) + 1 deliberately unverified (directory-listing) to exercise the badge + ranking penalty path live
Operating hours	Present on every record — added after review flagged that a "nearest facility" that is closed is a wrong answer
Payout values	Estimates, ₹/kg, explicitly labeled (payout_basis: "estimated"); UI disclaimer on every card: "Indicative market estimate — not an offer." No guaranteed or real-time rates claimed
Sourcing	Manual curation from publicly available / legally usable information. No scraping dependency — scraped rows lack the accepted-categories/payout/trust fields ranking needs, and ToS-restricted sources were excluded on principle

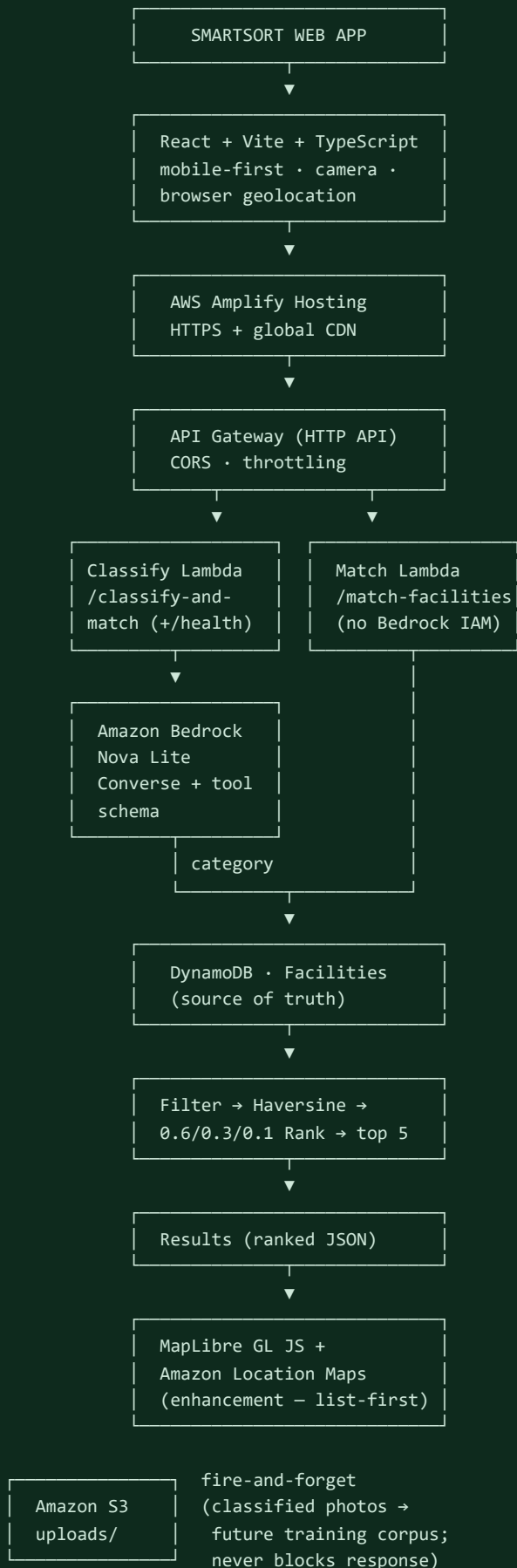
What this IS

Curated demo registry — sufficient, verified, honest seed data that makes the ranking engine and demo flow real. Fixed IDs; idempotent seed loader; versioned in git.

What this is NOT

A real-time production marketplace. No live availability, no live pricing, no guaranteed acceptance. Those require facility onboarding, verification workflows, and market-rate feeds (post-hackathon / long-term roadmap).

Final AWS Architecture



Service	Why it earns its place
Amplify Hosting	HTTPS + CDN + CI in minutes — HTTPS is a hard requirement for camera/geolocation on browsers
API Gateway HTTP API	Cheapest API entry point; CORS; throttling; single public boundary
Lambda ×2	Node 20 arm64, 512 MB, 15 s; split by responsibility (AI vs pure matching) with per-function least-privilege IAM
Bedrock Nova Lite	Multimodal classification, zero training/hosting, ap-south-1, per-request pricing
DynamoDB	PAY_PER_REQUEST; serverless source of truth for domain data no map provider has (accepted categories, payout estimates, verification)
S3	Photo archive + future training corpus; blocked public access; 90-day lifecycle; out of the synchronous path
Amazon Location	Maps tiles via API key (no Cognito); Places geocoding during curation; Routes optional for top-3 driving distance
SmartSort — Architecture Evolution	

Why the Current Architecture Is More Practical

Framing: the current architecture is **better aligned with the SmartSort MVP and the hackathon constraints** — not universally superior. A SageMaker pipeline with a trained waste model is the stronger *long-term* ML play; it is simply the wrong *5-day* play.

Dimension	Assessment
Product clarity	One sentence — “photograph waste, find where it's worth something” — maps 1:1 to screens and endpoints. The earlier framing buried the discovery bridge under the classifier.
Engineering complexity	One SAM stack, two functions, one table, one bucket. Every removed service (Cognito, CloudFront, async triggers) removed a failure mode and setup hours.
AI practicality	No dataset deadline, no model hosting, no \$4/hr idle billing; rationale output demos beautifully; validation chain handles the failure modes regex parsing couldn't.
Cost	Hackathon-spend bounded: pennies of Bedrock per photo; DynamoDB/S3/Lambda effectively free-tier at demo scale; compression reduces token cost; one Bedrock call per image by design.
Reliability	List-first product; labeled demo-location fallback; retry→fallback-model→manual-chips ladder; empty state designed out via seed coverage — the demo cannot dead-end in a blank screen.
Hackathon feasibility	Frontend already built and verified; backend is a template deploy + seed run away. Every remaining task is mechanical, not exploratory.
Security	Least-privilege per function (matching Lambda literally cannot invoke Bedrock); scoped S3 writes; map API key restricted; CORS tightened after Day 1.
Testing	The ranking formula, Haversine, compression, and classifier determinism are unit-tested — the parts judges probe are the parts with proofs.
Future scalability	The Lambda abstraction survives model swaps (Nova → fine-tuned custom); S3 corpus + override stream feed that fine-tuning; DynamoDB key design leaves room for geohash; nothing is thrown away post-hackathon.

Where the earlier plans were genuinely stronger

Custom-model classification offers a higher accuracy ceiling on domain-specific Indian waste streams; managed CloudFront distributions offer more control than Amplify's opinions; Cognito-native user accounts would jump-start retention features. These are acknowledged — and deliberately deferred, with the seams to add them back already in place.

SECTION 14

Reliability & Failure Handling

Failure	Detection	Automatic handling	User fallback
Bedrock failure / throttle	Invoke error / timeout	Retry ×2 → alternate model ID	Manual category chips → /match-facilities
Invalid tool output	Validation chain throw	Retry ×2 → fallback model → text-parse fallback	Manual chips
GPS denied / timeout	onerror / 10 s timeout	Typed failure status	“Use Demo Location” (labeled, Bengaluru) + map pin
Wrong classification	AI signal < 0.6 + user review	Gentle “not completely sure” nudge; top guesses	One-tap override → direct to matching (no AI re-run)
No matching facility	0 after category filter	Empty state with “try a larger search area”	Category change / radius expand (seed coverage makes this rare by design)
Map / tile failure	Renderer error / tile timeout	“Map unavailable” note	Ranked list persists — the product continues unaffected
Network failure	Fetch timeout / abort	Retryable error state (client retry ×2)	Retry button; hotspot; demo-mode canned responses; backup video
Oversized image	Post-compression size	Quality-stepping to q0.5; proceed with original if codec fails	Accepted best-effort; server limits remain the final guard
S3 write failure	Promise rejection (caught)	Silently skipped — never blocks or fails the response	None needed; archive is non-critical by design

Design invariant

The list is the core product; the map is an enhancement. Every screen remains fully functional with zero maps, zero GPS, and zero AI (manual chips) — the demo cannot hard-fail into a blank screen.

Demo-day protocol: cellular data instead of venue Wi-Fi · pre-loaded demo photos · demo-mode toggle armed · backup video on a second device · three successful full rehearsals before doors open · demo freeze at Day-5 noon: after that, only reliability fixes, no features.

SECTION 15

Security & Cost

Security

Private S3	Public access blocked on all four axes; uploads written by Lambda only
Least-privilege IAM	Per-function: classify = Bedrock invoke + Scan + scoped PutObject (uploads/*); match = Scan only — cannot touch Bedrock
CORS	Open only during Day-1 bring-up; locked to the Amplify domain thereafter
Map API key	Amazon Location key restricted to map APIs + referrer; no Cognito in the public path
Input validation	Category enum, confidence clamp, image type/size checks at the Lambda boundary
Coordinate validation	lat/lng range checks (−90...90 / −180...180) before distance math
Image validation	Base64-decodable, declared type, size ceiling before Bedrock invoke
Payload limits	API Gateway + Lambda request-size ceilings enforced server-side (final guard behind best-effort compression)
API throttling	Gateway default throttling/burst retained; protects the free tier from demo spikes
Secrets management	No secrets in the frontend; env vars per stage; map key is the only public-by-design credential and is referrer-scoped

Cost controls

Client compression	≤1024 px / ≈≤800 KB → fewer Bedrock image tokens per call
Bedrock once per image	Override path is AI-free by architecture, not by discipline
DynamoDB PAY_PER_REQUEST	Zero idle cost at rest; demo-scale reads are free-tier
Small dataset	14-item Scan is effectively free; no geospatial index to feed
S3 lifecycle	90-day expiration on uploads caps storage spend permanently
API throttling	Bounded request rate bounds worst-case spend
Billing alarm	CloudWatch alarm at a low threshold as a circuit breaker

MVP vs hardening

The left column is **MVP essentials** (in the SAM template / designed now). Rate-limiting per user, WAF, audit logging, and KMS at rest are **future hardening** — deliberately out of the 5-day scope and labeled as such.

Testing & Validation

Unit — 18 tests, all green

- **Ranking:** weights sum to 1.0; worked example A 0.780 > B 0.760; 5 km floor (>5 km flagged, never hidden); unverified penalty = exactly 0.1 and never filtered out; payout normalization vs best-eligible; tie-break by distance; top-5 cap; zero-payout safe
- **Haversine:** zero-distance identity; known Bengaluru pair ≈ 3.75 km; $0.01^\circ \approx 1.11$ km hand-check; symmetry
- **Compression:** ≤ 1024 px geometry rule; small images never upscaled; graceful fallback path when the codec fails
- **Mock classifier:** valid category from strict union; deterministic same-bytes-same-verdict; confidence in $[0,1]$; low-signal band exercised

Integration (backend phase)

- **API:** contract tests for both POSTs + /health from the real gateway; CORS verified from the deployed domain
- **Bedrock:** canned images → schema-valid verdicts; malformed-output → retry ladder; fallback model ID swap
- **DynamoDB:** seed loader idempotency; Scan → expected 14 items; per-category coverage counts
- **S3:** archive write succeeds; failure does not affect the response (by test, not by hope)

Frontend (verified in-browser)

- Camera/file input → preview → compression → staged loading
- Override chip → results *without* re-classification (log-verified invariant)
- Location: granted / denied / timeout → labeled demo fallback
- Result rendering: Best-match badge, pills, disclaimer on every card
- Map degradation → list-only with note

Real-device matrix (planned)

- Android Chrome + iOS Safari (camera capture attribute, permission prompts)
- Cellular network (not venue Wi-Fi) full-flow run
- Location permission grant *and* denial paths
- Native camera handoff from the capture input

AI evaluation — scoped honestly

Small internal test set — no generalized claims

A ~20–30 image internal set (a few per category, including dirty/damaged/multi-object cases) will be run through the deployed classifier to produce a confusion snapshot for the team. Results inform prompt iteration and demo photo selection. They do **not** support any claim of verified accuracy, and none will be made — the UI deliberately says “AI signal”.

MVP → Post-Hackathon → Long-Term

MVP (this hackathon)	Post-hackathon	Long-term vision
✓ AI classification (Bedrock Nova) ✓ Human confirmation / override ✓ Facility discovery (14-facility curated registry) ✓ Transparent 0.6/0.3/0.1 ranking ✓ Estimated value, clearly labeled ✓ Map (list-first, MapLibre + Location) ✓ AWS backend (SAM: API GW, Lambda×2, DynamoDB, S3) Explicitly out: accounts, payments, marketplace, notifications	○ User accounts (Cognito) + scan history ○ Facility onboarding & verification workflows ○ Live/verified payout rates (dealer feed) ○ Routes v2 driving distance for top-3 ○ Richer waste taxonomy (sub-categories) ○ Multilingual UI (regional languages) ○ Notifications & collection drives ○ Custom classifier trained on the S3 + override corpus	○ Waste listings by households & offices ○ Aggregation for apartments/societies ○ Recycler marketplace with discovery ○ Bidding on available materials ○ Verified market prices replace estimates ○ Payments in-platform ○ Logistics & multi-stop route optimization ○ Municipal waste-system integration ○ Carbon / impact tracking & reporting

Sequencing logic

The MVP deliberately builds only **Identify** → **Locate** → **Value** → **Discover** — the foundation the trading platform stands on. Registry, taxonomy, ranking, and valuation concepts are designed *now* to evolve into it: fixed facility IDs survive onboarding; the override stream becomes training data; per-category payout fields become market-rate slots. Nothing in the MVP pretends to be the marketplace; nothing in the marketplace requires rewriting the MVP.

Tradeoffs & Limitations (stated plainly)

Choice	Advantage	Limitation	Future improvement
Bedrock vs custom classifier	Zero training/hosting; demo cannot be blocked by an ML pipeline; strong on clutter/multi-object	Per-call cost at scale; domain accuracy on Indian waste streams unproven; confidence is a signal, not a calibration	Fine-tune on the S3 + override corpus; Custom Labels/SageMaker behind the same Lambda; calibrated confidence
Haversine vs driving distance	Sub-millisecond, zero dependencies, zero failure modes at 14 records	Straight-line distance ignores barriers/traffic — a 1.2 km walk can cross a highway	Routes v2 re-score for top-3 finalists (already specced as optional)
Curated vs real-time facilities	Verified, honest, demo-proof data with coverage guarantees	No live availability/hours changes; static snapshot	Facility onboarding + verification workflow; freshness timestamps surfaced
Estimated payout vs live price	Zero legal/commercial exposure; labeled honestly	Rates vary by dealer/quality; can disappoint vs expectation	Dealer price feed → verified rates; disclaimers step down as confidence rises
DynamoDB Scan vs geospatial index	Right-sized for ≤hundreds of facilities; trivially testable	Does not scale to thousands+ (scan cost grows linearly)	Geohash partition keys at hundreds; PostGIS/OpenSearch geo at thousands
Browser geolocation	No app install; one API; works on modern phones	Secure-context only; permission prompts; indoor accuracy varies; can be denied	Labeled demo fallback (shipped); manual map pin; IP-assist heuristics
“AI signal” wording	Zero unverifiable claims under questioning	Less impressive-sounding than “98% accurate”	A real evaluation on a proper labeled set would <i>earn</i> an accuracy statement
Small AI evaluation set	Honest, fast, actionable for prompt iteration	No statistical power; per-category picture incomplete	Grow the internal set from real usage; publish the confusion matrix internally

Final Technical Summary

The 60–90 second technical explanation (for judges)

“SmartSort is a **mobile-first responsive web application** — React, Vite and TypeScript on AWS Amplify — so users just open a URL; nothing to install. You photograph a waste item; the browser compresses it to under a megabyte; **Amazon Bedrock's Nova Lite** classifies the *material* — plastic, paper, metal, glass, e-waste, organic, or other — through the Converse API with a forced tool schema, and we validate that output at runtime. Classification is at most once per image: a separate `/match-facilities` endpoint serves human corrections without re-invoking the model. The confirmed category plus your location — with a clearly labeled demo fallback if GPS is denied — goes to a **Lambda** that reads a **DynamoDB** registry of 14 curated Bengaluru facilities, filters by accepted category, computes **Haversine** distance, and ranks with a transparent, unit-tested formula: 60% distance, 30% estimated value, 10% verification trust. Results render as ranked cards with hours, contacts, and an indicative ₹/kg that is always labeled an estimate — never an offer — plus a **MapLibre + Amazon Location** map that's an enhancement, never a dependency. Every failure has a designed fallback, and photos archive to **S3** fire-and-forget, building the training corpus for a custom model later. The whole backend is one SAM template.”

The 30–45 second non-technical explanation

“You know how you never know what to do with stuff you're throwing out — whether it can be recycled, or if it's worth money? SmartSort fixes that. You take a photo of the item. The app tells you what material it is, shows you the recycling and scrap centers near you that actually accept it, how far they are, when they're open, and roughly what it could be worth. You always confirm the category yourself — the AI is a helper, not the boss. Nothing to install: it just runs in your phone's browser.”

The one-breath product definition

SmartSort — an AI-powered responsive web platform for waste classification and recycling/reuse facility discovery:

Waste → Identification → Location → Facility → Value.

Final Go / No-Go Checklist

Frontend

- ☒ React+Vite+TS builds clean (strict TS)
- ☒ Capture → compress → classify → confirm → results flow E2E
- ☒ Override chips skip re-classification (verified)
- ☒ Location granted/denied/timeout → labeled demo fallback
- ☒ Mock API behind swappable interface; no mock data in components
- ☐ Mobile device-matrix pass (iOS Safari / Android Chrome)
- ☐ MapLibre + Location tiles wired (needs map phase)

AI

- ☒ Converse + tool schema + runtime validation designed
- ☒ “AI signal” terminology everywhere; no accuracy claims
- ☐ Live Bedrock call verified with real images (backend phase)
- ☐ Fallback model ID configured & tested
- ☐ Internal 20–30 image evaluation run

Backend

- ☒ SAM template + handler code specified
- ☒ Seed dataset finalized (14, coverage-proven)
- ☐ sam deploy --guided to ap-south-1
- ☐ Seed loader executed; /facilities returns 14
- ☐ CORS locked to deployed domain

AWS / Infra

- ☐ Location Map resource + restricted API key
- ☐ Amplify Hosting serving the built SPA (HTTPS)
- ☐ Billing alarm set; free-tier watch
- ☐ Env vars per stage (no secrets in git)

Security

- ☒ Least-privilege IAM per function (in template)
- ☒ S3 public access blocked + 90-day lifecycle
- ☐ Post-deploy IAM verification (match λ has no Bedrock)

Testing

- ☒ 18/18 unit tests (formula, geo, compression, classifier)
- ☒ Production build green
- ☐ API contract tests vs deployed gateway
- ☐ 3 full rehearsals on cellular

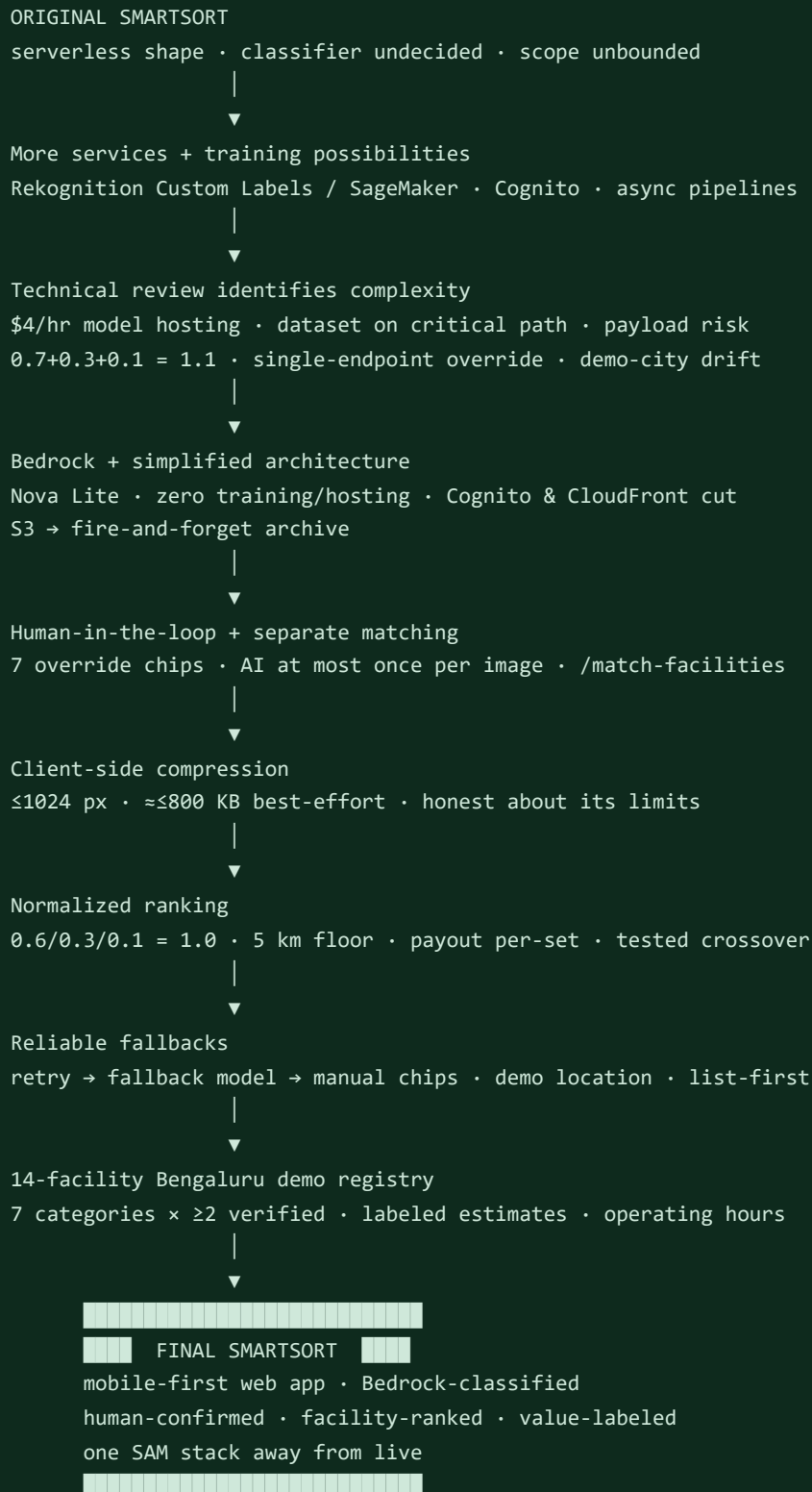
Demo & Docs

- ☐ Demo-mode toggle + backup video recorded
- ☐ Runbook (script + fallbacks) printed
- ☒ This architecture report
- ☐ README + repo cleanup for judges

Go / No-Go rule

Go = every ☒ above plus at minimum: deployed stack, seeded table, one live Bedrock image classified E2E, three rehearsals, backup video. If any of those five are missing on demo morning, present with the verified mock flow + video — never improvise on stage.

The Evolution, End to End



Original SmartSort → Technical Problems → Evidence-Based Refinement → Simplified AWS Architecture → Reliable MVP → Foundation for the Waste Trading Platform.

The architecture didn't grow up. It grew *down* — onto fewer, stronger foundations. That is what makes it defensible in a five-day hackathon

and defensible in a century.

22 / 22

SmartStart - Architecture Evolution